

C# FUNDAMENTALS

Tutorial & Practical Exercises By Engr. Nzenze Lovis

Data Types · Variables · Console I/O · DateTime

Section 1: Data Types in C#

What is a Data Type?

In C#, every piece of information your program works with has a type. A data type tells the compiler two things: what kind of data is being stored, and how much memory to reserve for it. Think of a data type like a labelled container — you wouldn't pour water into a bag designed for solid objects. The same principle applies in programming.

Primitive Data Types — The Building Blocks

The most fundamental data types in C# are called value types. They hold their data directly in memory and are the types you will use most frequently as a beginner.

Type	Size	Range / Values	Example
<code>int</code>	4 bytes	-2,147,483,648 to 2,147,483,647	<code>int age = 25;</code>
<code>double</code>	8 bytes	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	<code>double price = 9.99;</code>
<code>float</code>	4 bytes	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	<code>float temp = 36.6f;</code>
<code>char</code>	2 bytes	Single Unicode character	<code>char grade = 'A';</code>
<code>bool</code>	1 byte	true or false	<code>bool isStudent = true;</code>
<code>string</code>	Variable	Sequence of characters	<code>string name = "Alice";</code>
<code>decimal</code>	16 bytes	High-precision decimals	<code>decimal salary = 4500.00m;</code>
<code>long</code>	8 bytes	-9.2×10^{18} to 9.2×10^{18}	<code>long population = 8000000000L;</code>

NOTE

C# is a statically typed language. This means the type of every variable must be declared before use. The compiler will reject code that tries to assign the wrong type to a variable — this catches bugs early!

Type Conversion

Sometimes you need to convert between types. C# provides two approaches:

- Implicit conversion — happens automatically when there is no risk of data loss (e.g., int to double).
- Explicit conversion (casting) — you instruct the compiler to convert, accepting possible data loss (e.g., double to int).

```
// Implicit conversion - safe, no data lost
int myInt = 42;
double myDouble = myInt;    // 42.0 - perfectly fine

// Explicit cast - truncates the decimal portion
double pi = 3.14159;
int truncated = (int)pi;    // Result: 3 - decimal is dropped

// Using Convert class - useful for strings to numbers
string input = "100";
int parsed = Convert.ToInt32(input);    // Result: 100
int parsed2 = int.Parse(input);        // Alternative approach
```

Section 2: Variables in C#

Declaring and Initialising Variables

A variable is a named storage location in memory. In C#, you declare a variable by specifying its type followed by a name of your choosing. It is best practice to also initialise the variable (give it a starting value) at the point of declaration.

```
// Declaration only - the variable exists but has no meaningful value yet
int score;

// Declaration with initialisation - preferred approach
int score = 0;
string playerName = "Alice";
bool gameOver = false;
double accountBalance = 1500.75;
```

Naming Rules and Conventions

- Variable names must begin with a letter or underscore (_).
- Names are case-sensitive: score and Score are different variables.
- C# keywords (int, string, class, etc.) cannot be used as names.
- Use camelCase for local variables: playerName, totalScore, isActive.
- Use PascalCase for properties and methods: GetName(), TotalScore.

The var Keyword

C# supports type inference through the var keyword. The compiler determines the type from the assigned value. The variable is still strongly typed — var is not the same as dynamic.

```
var city = "Yaoundé";           // Inferred as string
var temperature = 28.5;         // Inferred as double
var population = 4000000;       // Inferred as int

// var is especially useful with long type names
var today = DateTime.Now;       // Much cleaner than writing DateTime today =
DateTime.Now;
```

Constants

When a value should never change throughout your program, declare it as a constant using the const keyword. This prevents accidental modification and improves code readability.

```
const double Pi = 3.14159265358979;
const string AppName = "StudentPortal";
const int MaxAttempts = 3;

// This line would cause a compile-time error:
// Pi = 3.0;    ← ERROR: Cannot assign to a constant
```

Section 3: Console Input & Output

Writing to the Console

The Console class in the System namespace provides methods to interact with the terminal window. You will use this for both displaying results and collecting user input.

```
// Console.Write - prints text WITHOUT moving to a new line
Console.Write("Enter your name: ");

// Console.WriteLine - prints text AND adds a line break at the end
Console.WriteLine("Hello, World!");

// String interpolation - embed variables directly inside a string using $
string name = "Alice";
int age = 21;
Console.WriteLine($"Name: {name}, Age: {age}");

// String.Format - an older but equally valid approach
Console.WriteLine("Name: {0}, Age: {1}", name, age);
```

Reading from the Console

`Console.ReadLine()` pauses execution and waits for the user to type a value and press Enter. It always returns a string, so you must convert the input if you need a number or another type.

```
// Reading a string – no conversion needed
Console.Write("Enter your name: ");
string name = Console.ReadLine();

// Reading an integer – must convert from string
Console.Write("Enter your age: ");
int age = int.Parse(Console.ReadLine());

// Safer approach using int.TryParse – avoids crashes on bad input
Console.Write("Enter a number: ");
if (int.TryParse(Console.ReadLine(), out int number))
{
    Console.WriteLine($"You entered: {number}");
}
else
{
    Console.WriteLine("That was not a valid number.");
}
```



`int.Parse()` will throw a `FormatException` if the user types something that is not a number. For production code, always prefer `int.TryParse()`. For these exercises, `int.Parse()` is acceptable, but note the risk.

Section 4: Working with DateTime

The DateTime Struct

C# provides a built-in `DateTime` struct (in the `System` namespace) for representing dates and times. It is one of the most commonly used types in real-world applications — from logging events to calculating deadlines and ages.

```
// Get the current date and time
DateTime now = DateTime.Now;
Console.WriteLine(now);    // e.g. 18/03/2026 14:35:22

// Get only the current date (time portion is midnight)
DateTime today = DateTime.Today;

// Create a specific date
DateTime birthday = new DateTime(2000, 6, 15);    // Year, Month, Day

// Parse a date from a string the user has typed
string input = "15/06/2000";
DateTime parsed = DateTime.ParseExact(input, "dd/MM/yyyy",
    System.Globalization.CultureInfo.InvariantCulture);
```

Practical Exercises

The following four exercises are designed to assess your understanding of data types, variables, and console input/output. Read each specification carefully before writing a single line of code. Plan your variable types and conversions first.

GRADING

Each exercise carries 25 marks. Marks are awarded for: correct data types (8), working logic (10), output formatting (4), and clean variable naming (3).

Exercise 1: Student Profile Card

Background

You have been asked to build a simple intake tool for a university registration desk. The application must collect several pieces of information from a new student, store them in appropriately typed variables, and display a neatly formatted profile card.

Requirements

- Prompt the user to enter their full name (string).
- Prompt for their student ID number — a whole number up to 999999 (int).
- Prompt for their current GPA to two decimal places (double).
- Prompt for their faculty initial — a single letter such as S for Science (char).
- Prompt whether they are a full-time student (accept 'true' or 'false') (bool).
- Prompt for their annual tuition fee (decimal — use this type specifically for monetary values).
- Display a formatted profile card using string interpolation.

Expected Output Format

STUDENT PROFILE CARD	
Name	: Alice Mbeki
Student ID	: 204571
GPA	: 3.72
Faculty	: S
Full-Time	: True
Tuition	: XAF 850,000.00

Critical Thinking Questions

- a) Why is decimal preferred over double for storing a tuition fee?

- b) What would happen if you stored the student ID as a string instead of an int? Give one advantage and one disadvantage.
- c) `Console.ReadLine()` always returns a string. What two steps are needed to store the GPA correctly?

Exercise 2: Temperature Unit Converter

Background

A local weather station needs a quick conversion tool. The application must accept a temperature value and a source unit from the user, then convert and display the equivalent values in all three common units: Celsius, Fahrenheit, and Kelvin.

Conversion Formulae

From Celsius	From Fahrenheit	From Kelvin
$^{\circ}\text{F} = (^{\circ}\text{C} \times 9/5) + 32$ $\text{K} = ^{\circ}\text{C} + 273.15$	$^{\circ}\text{C} = (^{\circ}\text{F} - 32) \times 5/9$ $\text{K} = (^{\circ}\text{F} - 32) \times 5/9 + 273.15$	$^{\circ}\text{C} = \text{K} - 273.15$ $^{\circ}\text{F} = (\text{K} - 273.15) \times 9/5 + 32$

Requirements

- Prompt the user to enter a numeric temperature value. Use double for precision.
- Prompt for the unit: "C", "F", or "K". Store as a string, but compare using char or use string methods.
- Use an if / else if / else structure to handle each case.
- Display all three unit values rounded to two decimal places using `Math.Round()`.
- Handle invalid unit input gracefully by displaying a clear error message.

Challenge Extension (Bonus)

Extend the program to check whether the computed Kelvin value is physically valid. Temperature in Kelvin cannot be negative (absolute zero is 0 K). If the user's input would result in a negative Kelvin value, display a warning.

Critical Thinking Questions

- d) Why is double more appropriate than float for this exercise?
- e) Integer division is a common bug in C#. Explain what `9/5` evaluates to in C#, and how you would fix it.
- f) What does `Math.Round(3.145, 2)` return? Research banker's rounding and explain how to force standard rounding.

Exercise 3: Simple Shopping Receipt

Background

A small convenience store needs a basic point-of-sale application. The application must collect details for three items purchased, calculate the subtotal, apply a VAT rate, and print a formatted receipt to the console.

Requirements

- Collect the name, unit price (decimal), and quantity (int) for three separate items.
- Compute each line total: $\text{lineTotal} = \text{unitPrice} \times \text{quantity}$.
- Compute the subtotal (sum of all three line totals).
- Apply a VAT rate of 19.25% (store the rate as a named const).
- Compute and display the VAT amount and the grand total.
- Format all monetary values to two decimal places.
- Display a complete, readable receipt as shown below.

Expected Output Format

```
=====
          CONVENIENCE STORE RECEIPT
=====
Item                Qty   Price   Total
-----
Mineral Water (500ml)  3   500.00  1500.00
Bread (Sliced)        2   350.00   700.00
Orange Juice (1L)     1   950.00   950.00
-----
Subtotal:                3150.00
VAT (19.25%):            606.38
=====
GRAND TOTAL:                3756.38
=====
          Thank you for shopping with us!
```

Critical Thinking Questions

- You are working with money. Explain why using decimal instead of double is important here and what kind of errors double can introduce.
- The VAT rate is declared as a const. What would go wrong if another developer changed it to a regular variable? Why does const improve code reliability?
- How would you modify the program to accept any number of items (not just three), without knowing the count in advance? (You do not need to implement this — describe your approach.)

Exercise 4: Age Calculator — DateTime Challenge

Background

A healthcare clinic requires a patient intake tool. The application must collect a patient's date of birth in a specific format, validate that the input is a real date, calculate the patient's exact age in years, and classify them into an age group. This exercise tests your ability to work with the DateTime struct and type conversion.

Requirements

- Prompt the user to enter their date of birth in the format DD/MM/YYYY (e.g., "15/06/1990").
- Parse the input into a DateTime variable using DateTime.ParseExact() with the format string "dd/MM/yyyy".
- Validate the input: if the string cannot be parsed, display an error message and exit gracefully (use DateTime.TryParseExact()).
- Validate that the date of birth is not in the future.
- Calculate the patient's exact age in years, correctly accounting for whether their birthday has occurred yet this year.
- Classify the patient using the age group table below.
- Display a complete summary as shown in the expected output.

Age Range	Classification	Notes
0 – 12 years	Child	Paediatric ward referral
13 – 17 years	Teenager	Guardian consent required
18 – 64 years	Adult	Standard intake procedure
65+ years	Senior	Priority queue; geriatric review

Expected Output Format

```
CLINIC PATIENT INTAKE TOOL

Enter date of birth (DD/MM/YYYY): 15/06/1990

Date of Birth : 15 June 1990
Age           : 35 years
Classification: Adult
Note          : Standard intake procedure
```

```
// Invalid input scenario:
Enter date of birth (DD/MM/YYYY): 30/02/1990
ERROR: '30/02/1990' is not a valid date. Please check the day and month.

// Future date scenario:
Enter date of birth (DD/MM/YYYY): 01/01/2030
ERROR: Date of birth cannot be in the future.
```

Critical Thinking Questions

- Explain the difference between DateTime.Parse() and DateTime.ParseExact(). When would each be appropriate?
- Why must you subtract one from the calculated age in some cases? Describe the scenario where omitting this check produces the wrong answer.

- l) The format string "dd/MM/yyyy" uses specific casing. What would change if you used "DD/MM/YYYY" or "dd/mm/yyyy" instead?
- m) How would you extend this program to also display the number of days until the patient's next birthday?

Quick Reference Summary

Concept	Key Class / Keyword	Common Pitfall
Data Types	<code>int, double, decimal, bool, char, string</code>	Using double for money — use decimal instead
Variables	<code>var, const</code>	Forgetting to initialise before use
Console Output	<code>Console.Write / WriteLine</code>	Forgetting \$ for string interpolation
Console Input	<code>Console.ReadLine()</code>	Forgetting to convert the string return value
Type Conversion	<code>int.Parse(), Convert.ToInt32(), TryParse()</code>	Using Parse() without error handling
DateTime	<code>DateTime.Now, ParseExact(), TryParseExact()</code>	Not accounting for birthday in age calculation

Good luck! Remember: a good programmer writes code that humans can read, not just code that machines can run.